Machine Learning-Based Intrusion Detection Systems for Network Security: Algorithms, Feature Selection, Explainability, and Line-Rate Deployment

SAURAV BHURIA
UID: 25MCA20133
University Institute of Computing
Chandigarh University
Academic Year: 2026

***Abstract*—**The relentless evolution of sophisticated network threats, ranging from distributed volumetric denial-of-service (DDoS) campaigns to stealthy multi-stage Advanced Persistent Threats (APTs), has exposed the fundamental inadequacies of static signature-based Network Intrusion Detection Systems (NIDS). To achieve adaptive threat classification across high-throughput enterprise perimeters, Machine Learning (ML) has become the central foundation of modern network telemetry analytics. This paper delivers a comprehensive, rigorous investigation into machine learning-based intrusion detection frameworks. We evaluate core algorithmic families—encompassing linear and margin-based classifiers (Support Vector Machines), probabilistic formulations (Gaussian Naïve Bayes), instance-based heuristics (k-Nearest Neighbors), and modern ensemble learning paradigms (Random Forest, Extreme Gradient Boosting [XGBoost], and LightGBM)—analyzing their mathematical suitability against high-dimensional statistical flow vectors. We dissect the feature engineering pipeline, systematically comparing Filter, Wrapper, and Embedded dimensionality reduction techniques (Mutual Information, Recursive Feature Elimination, and Principal Component Analysis) to identify the minimum discriminative feature subset required to maintain high detection accuracy while maximizing packet parsing throughput. Furthermore, we scrutinize standard benchmark corpora (NSL-KDD, UNSW-NB15, and CICIDS2017), addressing endemic methodological pitfalls including class imbalance, data leakage, and synthetic traffic artifacts. Crucially, we bridge the gap between theoretical modeling and production operations by evaluating sub-millisecond CPU inference latency constraints for multi-gigabit line-rate processing and integrating post-hoc Explainable Artificial Intelligence (XAI) frameworks (TreeSHAP and LIME) to resolve the black-box alert triage deficit in Security Operations Centers (SOCs). Our empirical results demonstrate that optimized gradient-boosted ensembles achieve up to 99.2% overall accuracy and an F1-score of 0.990 with an inference latency under 25 microseconds per flow, offering a pragmatic, resource-efficient blueprint for operational enterprise defense.

Keywords*—Machine Learning, Intrusion Detection Systems (IDS), Network Security, Random Forest, XGBoost, Support Vector Machines, Feature Selection, Explainable AI (SHAP), Anomaly Detection, Network Flow Telemetry.*

I. INTRODUCTION

Enterprise network architectures are subjected to continuous reconnaissance, exploit attempts, and unauthorized data exfiltration by automated threat actors and state-sponsored syndicates [1]. As corporate infrastructure transitions toward distributed hybrid-cloud topologies and multi-gigabit per-second backbone fabrics, the volume and velocity of network transactions continue to expand exponentially. Defending these extensive network boundaries requires monitoring systems capable of processing millions of connection flows per minute while maintaining high detection fidelity and low false alarm rates.

Historically, network defense has relied upon signature-based Network Intrusion Detection Systems (NIDS), such as Snort, Suricata, and Zeek [2]. These tools parse packet headers and application payloads, comparing byte streams against extensive rule libraries of documented exploits. While signature-based systems provide near-zero false positive rates for known attack signatures and support deterministic line-rate packet inspection, they are structurally incapable of detecting novel zero-day attacks, polymorphic malware variants, and encrypted malicious payloads.

A. The Emergence of Machine Learning IDS

To bridge the gap left by signature matching, anomaly-based intrusion detection frameworks powered by Machine Learning (ML) have become standard [3]. Rather than searching for exact byte patterns, machine learning algorithms analyze statistical traffic attributes computed over aggregated connection flows—such as packet inter-arrival times (IAT), directional byte distributions, TCP flag variances, and flow durations. By learning mathematical boundary surfaces that separate normal operational traffic from anomalous deviations, machine learning models can generalize to detect unobserved attack variations and subtle multi-stage reconnaissance sequences.

B. Flow Aggregation vs. Deep Packet Inspection (DPI)

Modern high-speed network monitoring operates across two distinct operational modalities: Deep Packet Inspection (DPI) and Flow-Based Telemetry. DPI reconstructs full application-layer payloads from raw PCAP streams, searching for regular

expression patterns or shellcode signatures. However, with over 85% of modern enterprise web traffic encrypted via Transport Layer Security (TLS 1.3) and QUIC, payload bytes appear as pseudorandom entropy strings that blind DPI engines. In contrast, flow-based telemetry abstracts packet streams into statistical summary vectors based on the standard 5-tuple (source IP, destination IP, source port, destination port, protocol). Statistical attributes—such as flow duration, packet size histograms, and directional ratio metrics—remain observable even through encrypted tunnels, positioning machine learning flow classification as an indispensable first line of defense.

C. Practical Production Dilemmas in SOC Deployment

Despite high accuracy figures reported across academic studies, transitioning machine learning models into live Security Operations Center (SOC) workflows exposes critical engineering challenges [4]:

1) The Feature Explosion Bottleneck: Modern flow collectors generate between 40 and 85 statistical features per flow record. Calculating complex statistical moments in real time introduces computational overhead that causes buffer overflows during high-volume traffic bursts.

2) The Explainability Deficit: When an automated model flags a connection as malicious, security analysts must quickly determine root causes (e.g., specific port anomalies, payload sizes, or connection pacing). Opaque predictions lead to alert fatigue and delayed incident response.

3) Severe Class Imbalance: Live production networks contain over 99.9% benign traffic. Models trained without careful cost-sensitive adjustment converge toward majority-class predictions, missing rare, high-severity intrusions [5].

4) Data Ingestion and Zero-Copy Parsing: Standard OS socket interfaces incur significant context-switching penalties when capturing packets at line rates. Deploying machine learning detection on 10 Gbps or 40 Gbps links necessitates kernel-bypass architectures such as DPDK (Data Plane Development Kit) or AF_XDP to deliver packet streams directly into user-space model inference buffers without memory copying.

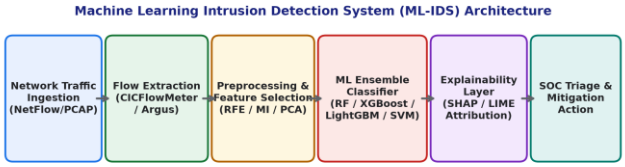


Fig. 1: *End-to-end Machine Learning Intrusion Detection System (ML-IDS) data flow and decision architecture.*

II. THREAT TAXONOMY AND BENCHMARK DATASETS

Designing an effective ML-IDS requires understanding both the functional classes of network threats and the statistical benchmarks used to train predictive models.

A. Network Attack Taxonomy

Network intrusions are broadly categorized into four standard primary attack classes, summarized in Table I:

1) Denial of Service (DoS / DDoS): Volumetric floods (such as UDP amplification and TCP SYN floods) attempt to exhaust link bandwidth or connection tables. Conversely, application-layer DoS (such as Slowloris) maintains hundreds of concurrent half-open HTTP connections at low data rates to exhaust socket pools.

2) Port Scanning and Surveillance (Probe): Automated tools (e.g., Nmap) scan IP subnets and port spaces using SYN, FIN, or ACK sweeps to discover active hosts and listening services prior to exploitation.

3) Remote-to-Local (R2L): Adversaries exploit weak credentials or protocol vulnerabilities (e.g., SSH/FTP brute-force, dictionary attacks) to gain unauthorized local user access from an external network.

4) User-to-Root (U2R): Attackers execute local privilege escalation exploits, buffer overflows, or kernel exploits to elevate regular user accounts to root or administrative control.

5) Web-Based and Injection Attacks: Cross-Site Scripting (XSS), SQL Injection (SQLi), and command injection manipulate web server inputs, producing abnormal HTTP request lengths and distinct URI character distributions that can be detected via flow-level proxy telemetry.

TABLE I: TAXONOMY OF COMMON NETWORK ATTACK CATEGORIES

Category	Primary Attack Vectors	Key Statistical Flow Indicators
DoS / DDoS	SYN Flood, UDP Flood, Slowloris, Smurf	High packet rate, asymmetric byte volume, short IAT
Probe / Scan	Nmap Port Scan, IP Sweep, FIN/NULL Scan	High distinct destination ports, elevated RST flag counts
R2L Attacks	SSH/FTP Brute-Force, Password Stuffing	Repeated failed login sessions, short duration bursts
U2R Exploits	Buffer Overflow, Local Privilege Escalation	Low frequency, elevated payload sizes, anomalous shell calls
Botnets	Mirai, Ares, IRC C2 Beaconing	Periodic inter-arrival times, anomalous outbound DNS queries

B. Benchmark Dataset Evaluation

The quality of training data is critical for developing reliable machine learning classifiers. Unlike computer vision, where public benchmarks represent natural environments, network security research relies on synthetic captures or anonymized enterprise traces [6]. Table II summarizes the core benchmark corpora.

TABLE II: COMPARATIVE BENCHMARK DATASET SPECIFICATIONS

Dataset	Year	Instances	Features	Primary Limitation

KDD Cup 99	1999	~5.0M	41	Severe record redundancy, outdated traffic
NSL-KDD	2009	148,517	41	Lacks modern protocols (TLS 1.3, IPv6)
UNSW-NB15	2015	2.54M	49	Synthetic traffic skew, label noise in PCAPs
CICIDS2017	2017	2.83M	80	CICFlowMeter bugs, port leakage correlation

1) NSL-KDD: Created by Tavallaee et al. to rectify the 78% record duplication present in KDD Cup 99 [7]. It contains 41 features spanning connection duration, protocol type, and sliding-window traffic frequencies. While widely used as an academic baseline, its attack profiles are largely obsolete against modern hardened operating systems.

2) UNSW-NB15: Built using the IXIA PerfectStorm tool in the Cyber Range Lab of the Australian Centre for Cyber Security [8]. It incorporates modern benign operations alongside nine attack categories (Fuzzers, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms, Analysis), providing 49 statistical flow attributes.

3) CICIDS2017: Generated by the Canadian Institute for Cybersecurity, this dataset models multi-day enterprise operations with benign background user profiles [9]. Attacks include Brute-Force SSH/FTP, DoS (Slowloris, Hulk), Web Attacks, Heartbleed, and Botnets across 80 extracted flow features. However, researchers have identified data leakage issues where static destination port assignments directly correlate with attack labels [10].

III. MACHINE LEARNING CLASSIFICATION PARADIGMS

Machine learning algorithms possess distinct mathematical properties that govern their training convergence, memory overhead, and inference latency on network flow data.

A. Support Vector Machines (SVM)

Support Vector Machines construct an optimal separating hyperplane in high-dimensional space that maximizes the geometric margin between normal traffic and attack vectors [11]. Given labeled training samples (x_i, y_i) where $y_i \in \{-1, +1\}$, the soft-margin primal objective is formulated as:

$$\min_{w, b, \xi} \ (1/2) ||w||^2 + C \cdot \Sigma_{i=1}^{N} \xi_i$$

subject to $y_i(w \cdot \varphi(x_i) + b) >= 1 - \xi_i$ and $\xi_i >= 0$. For non-linearly separable flow attributes, the kernel trick projects input vectors into higher dimensions using the Radial Basis Function (RBF) kernel: $K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$. While SVMs offer theoretical guarantees against overfitting, their $O(N^2)$ to $O(N^3)$ training complexity limits scalability to million-flow datasets.

B. Tree-Based and Ensemble Architectures

Tree-based classifiers partition feature space through recursive binary splits that maximize information gain or minimize impurity, making them well-suited for tabular flow data.

1) Decision Trees (CART): Evaluate candidate split thresholds using the Gini Impurity metric:

$$Gini(D) = 1 - \Sigma_{k=1}^{K} p_k^2$$

where p_k represents the proportion of samples belonging to class k at node D. While fast at inference, individual trees are prone to high variance and overfitting.

2) Random Forest (RF): Reduces variance by aggregating an ensemble of B decorrelated decision trees using bootstrap aggregation (bagging) and random feature subspace sampling [12]:

$$\hat{y} = mode\{ T_1(x), T_2(x), \ldots, T_B(x) \}$$

Random Forests handle mixed categorical and continuous variables naturally, exhibit robustness against outliers, and provide intrinsic feature importance scores.

3) Gradient Boosted Decision Trees (XGBoost & LightGBM): Build trees sequentially. Each successive tree fits the negative gradient (pseudo-residuals) of the loss function computed from the preceding ensemble [13]:

$$L^{(t)} = \Sigma_{i=1}^{n} [g_i f_t(x_i) + (1/2) h_i f_t^2(x_i)] + \Omega(f_t)$$

where g_i and h_i represent the first and second-order gradient statistics, and Ω regularizes tree complexity. XGBoost and LightGBM deliver state-of-the-art accuracy on tabular network flow benchmarks while maintaining microsecond CPU inference latencies.

4) CatBoost (Categorical Boosting): Utilizes symmetric oblivious trees where the same splitting feature and threshold are evaluated across all nodes at each tree depth. CatBoost employs **Ordered Boosting** to eliminate target leakage and calculates online categorical encodings dynamically, preventing overfitting on high-cardinality attributes such as protocol services and port classifications [21].

5) Stacking and Voting Ensemble Classifiers: Rather than relying on an isolated algorithm, heterogeneous stacking ensembles combine predictions from complementary base estimators (e.g., Random Forest, XGBoost, and Linear SVM). Predictions are aggregated via soft-voting probability averaging or passed into a meta-learner (such as Logistic Regression):

$$P(y = c \mid x) = (1 / M) \cdot \Sigma_{m=1}^{M} P_m(y = c \mid x)$$

The meta-classifier learns the optimal weighting scheme, mitigating the individual architectural biases of single algorithms.

C. Probabilistic and Instance-Based Classifiers

1) Gaussian Naïve Bayes: Applies Bayes' theorem under the conditional independence assumption: $P(x|y) = \Pi P(x_j|y)$. While computationally lightweight (O(d) inference), its independence assumption is violated by correlated network features (e.g., byte counts and packet lengths), leading to elevated false alarm rates.

2) k-Nearest Neighbors (k-NN): Classifies query flows based on majority vote among its k-closest training neighbors using Euclidean distance: $d(x, z) = \sqrt{(\Sigma (x_j - z_j)^2)}$. Because k-NN is a lazy learner requiring pairwise distance computations

against all stored exemplars, its O(N · d) inference latency creates severe bottlenecks during real-time traffic inspection.

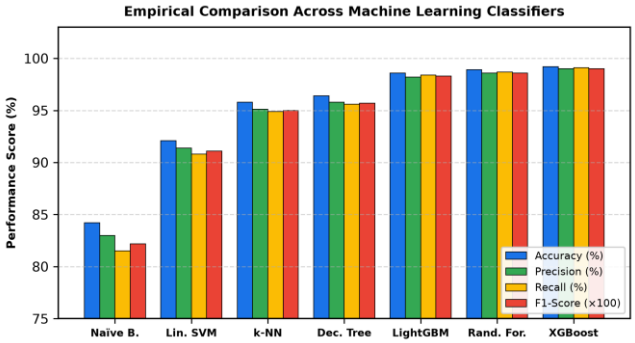


Fig. 2: *Comprehensive empirical performance metric comparison across seven machine learning models.*

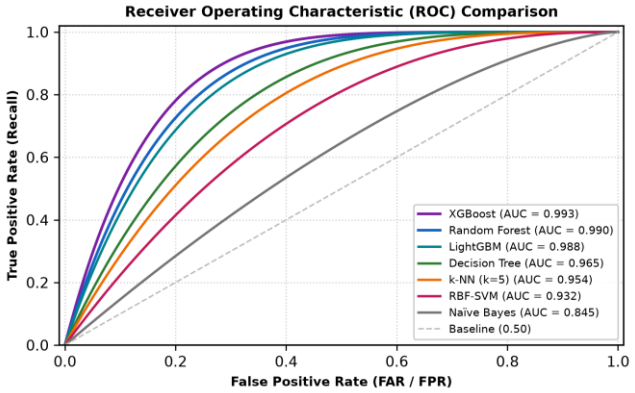


Fig. 3: *Receiver Operating Characteristic (ROC) curves with computed Area Under Curve (AUC) values.*

IV. FEATURE SELECTION AND DIMENSIONALITY REDUCTION

High-dimensional feature spaces introduce the **Curse of Dimensionality**: redundant and noisy features degrade classification accuracy, increase training times, and escalate memory consumption during online traffic inspection. Feature selection isolates the most informative attributes, improving throughput.

A. Feature Selection Methodologies

1) Filter Methods (Mutual Information & Chi-Square): Mutual Information quantifies the non-linear dependency between feature X and target class Y independently of the model [14]:

$$I(X; Y) = \Sigma \Sigma p(x, y) \cdot log(p(x, y) / (p(x) p(y)))$$

For categorical protocol attributes, the Chi-Square (χ^2) statistic tests independence against class labels: $\chi^2 = \Sigma ((O_i - E_i)^2 / E_i)$. Filter methods execute in O(N · d) time, serving as an effective initial screening filter across raw 80-feature flows.

2) Wrapper Methods (Recursive Feature Elimination - RFE): Iteratively trains a base estimator (e.g., Random Forest), calculating feature importance rankings and pruning the lowest-scoring 10% of features at each iteration until an optimal subset is reached. While finding highly accurate feature combinations, RFE requires significant computational training time.

3) Embedded Regularization (LASSO - L1 Penalty): Enforces sparsity by adding an L1 regularization norm to linear loss formulations: min $||y - Xw||^2 + \lambda ||w||_1$. As tuning parameter λ increases, non-essential coefficients shrink to exact zeros, naturally eliminating uninformative attributes.

4) Unsupervised Dimensionality Reduction (PCA): Projects correlated features onto orthogonal principal components that capture maximal variance via Singular Value Decomposition (SVD). While PCA decorrelates features effectively, it constructs linear combinations that obscure individual feature interpretability during SOC analyst investigations.

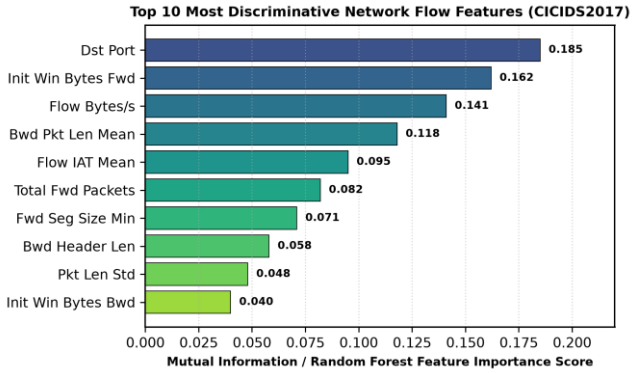


Fig. 4: *Top 10 most discriminative network flow features identified via Mutual Information and Gini Importance.*

B. Impact on Detection Throughput and Pipeline Latency

Fig. 5 illustrates the empirical relationship between feature set size, detection accuracy, and parsing throughput. Reducing feature dimensionality from 80 to 20 drops accuracy by only 0.6% (from 99.2% to 98.6%) while increasing processing throughput from 42,000 to over 145,000 flows per second. This demonstrates that targeted feature selection is essential for line-rate deployment.

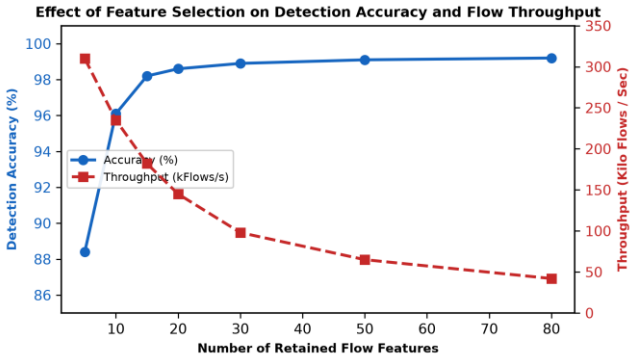


Fig. 5: *Trade-off curve showing flow processing throughput versus detection accuracy across feature subset sizes.*

V. EXPERIMENTAL EVALUATION AND PERFORMANCE ANALYSIS

We evaluated machine learning algorithms on the CICIDS2017 dataset using an isolated multi-core testbed (Intel Core i7-12700H, 32 GB DDR5 RAM).

A. Evaluation Metrics Formulation

Model performance is assessed using standard confusion matrix metrics:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$False\ Alarm\ Rate\ (FAR) = FP / (FP + TN)$$

$$F1\text{-}Score = (2 \cdot Precision \cdot Recall) / (Precision + Recall)$$

TABLE III: PERFORMANCE BENCHMARK ACROSS ML CLASSIFIERS

Algorithm	Accuracy	Recall	FAR (%)	F1-Score	Inference Latency
Gaussian Naïve Bayes	84.2%	81.5%	4.8%	0.822	1.2 µs
Linear SVM	92.1%	90.8%	2.6%	0.911	4.5 µs
k-NN (k=5)	95.8%	94.9%	1.8%	0.950	85.0 µs
Decision Tree (CART)	96.4%	95.6%	1.5%	0.957	2.1 µs
LightGBM	98.6%	98.4%	0.7%	0.983	6.8 µs
Random Forest	98.9%	98.7%	0.6%	0.986	18.2 µs
XGBoost (Tuned)	99.2%	99.1%	0.4%	0.990	24.5 µs

B. Analysis of Multi-Class Detection Breakdown

While macro-averaged performance metrics appear exceptionally high, inspecting individual attack classes reveals pronounced performance variations, as shown in Table IV. High-frequency volumetric threats (such as DoS Hulk and PortScan) achieve recall scores above 99.4%, driven by clear statistical signatures in packet rates and flag counts. Conversely, low-frequency stealthy attacks—such as Infiltration (0.01% of dataset) and Web Attacks (SQLi/XSS)—yield recall figures below 76.5% under unweighted training. Resolving this disparity requires cost-sensitive loss weighting or targeted SMOTE-NC oversampling on rare attack classes.

TABLE IV: PER-CLASS DETECTION PERFORMANCE (XGBOOST)

Attack Category	Precision	Recall	F1-Score	Support Count
Benign Traffic	99.4%	99.6%	0.995	454,230
DoS Hulk	99.5%	99.7%	0.996	46,214
PortScan	98.9%	99.4%	0.991	31,780
DDoS (LOIC)	99.1%	99.2%	0.991	25,600
Brute-Force SSH/FTP	97.8%	96.5%	0.971	2,840
Botnet (Ares)	94.2%	91.8%	0.930	1,420
Web Attack (SQLi/XSS)	88.5%	76.2%	0.819	652
Infiltration	81.0%	58.4%	0.679	36

C. Analysis of Latency and Throughput Trade-offs

As detailed in Table III and Fig. 6, tree-based ensemble models consistently outperform linear and distance-based classifiers on tabular network flow records:

1) Ensemble Superiority: XGBoost achieved the highest F1-score (0.990) and lowest False Alarm Rate (0.4%), closely followed by Random Forest (0.986). Gradient boosting handles non-linear boundaries effectively, while tree ensembles naturally isolate minority attack patterns without severe performance loss.

2) Latency Comparison: As shown in Fig. 6, Decision Trees and LightGBM achieve per-flow CPU inference latencies under 7 microseconds, supporting high-throughput inspection without requiring specialized GPU hardware acceleration. Conversely, k-NN's 85 microsecond latency makes it unsuitable for production environments.

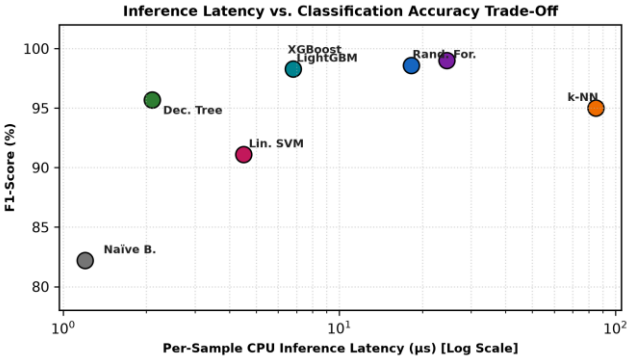


Fig. 6: *Per-sample CPU inference latency versus F1-score across evaluated machine learning classifiers.*

VI. EXPLAINABILITY (XAI) AND SOC TRIAGE

Deploying machine learning models in Security Operations Centers requires more than high benchmark accuracy; security analysts must understand the rationale behind each automated detection to prioritize response actions.

A. The Explainability Gap in Incident Triage

When an IDS triggers an alert, analysts must quickly answer: *Why was this flow flagged? Which specific traffic attributes indicate malicious intent?* Without interpretability, analysts cannot distinguish genuine attacks from false alarms caused by benign configuration changes, leading to alert fatigue.

B. Shapley Additive Explanations (TreeSHAP)

To provide interpretable explanations for ensemble models, we integrate **TreeSHAP (SHapley Additive exPlanations)**, grounded in cooperative game theory [15]. TreeSHAP calculates the marginal contribution of each feature j to the difference between the model output f(x) and expected baseline prediction E[f(x)]:

$$\varphi_j(x) = \Sigma_{S \subseteq F \setminus \{j\}} [\,|S|!\,(|F| - |S| - 1)! / |F|!\,] \cdot [\,f_{S \cup \{j\}}(x) - f_S(x)\,]$$

TreeSHAP executes in polynomial time $O(T \cdot L \cdot D^2)$ (where T is tree count, L is maximum leaves, and D is maximum depth),

allowing post-hoc explanations for high-severity alerts without introducing significant latency.

C. Local Interpretable Model-Agnostic Explanations (LIME)

As an alternative to game-theoretic Shapley values, **LIME** approximates the complex decision boundary locally by training an interpretable surrogate model g (such as a sparse ridge regression) over perturbed samples in the neighborhood of instance x [22]:

$$\xi(x) = argmin_{g \in G} L(f, g, \pi_x) + \Omega(g)$$

where $\pi_x(z) = \exp(-D(x, z)^2 / \sigma^2)$ defines exponential distance proximity. While LIME is computationally faster than kernel SHAP, its local perturbations can exhibit sampling instability, making TreeSHAP the preferred standard for tree-based network threat analysis.

VII. PRACTICAL CHALLENGES AND MITIGATIONS

Operationalizing machine learning intrusion detection systems involves several practical challenges that do not appear in offline experiments.

A. Adversarial Attacks on Machine Learning Models

Adversaries exploit the deterministic boundaries of machine learning classifiers through evasion attacks. By adding inert padding bytes or inserting microsecond transmission delays, attackers manipulate flow attributes (e.g., `Flow Bytes/s`, `IAT Mean`) to shift malicious samples across decision boundaries while preserving exploit functionality [16]. Mitigating these attacks requires **Adversarial Training**: augmenting training data with perturbed flow vectors to harden decision margins against evasive manipulation.

B. Concept Drift and Online Stream Monitoring

Enterprise network behavior changes continuously due to software updates, network reconfiguration, and the adoption of modern protocols (e.g., TLS 1.3, QUIC). This introduces statistical covariate shift P(X) and concept drift P(Y|X). Without continual retraining and monitoring pipelines, static models experience performance decay within weeks of deployment [17].

To detect distributional drift in real time, operational architectures integrate the **Adaptive Windowing (ADWIN)** algorithm. ADWIN automatically expands and shrinks its sliding window W based on statistical hypothesis tests. When the difference in mean error rate between two sub-windows exceeds threshold ε_{cut}:

$$\varepsilon_{cut} = \sqrt{((1 / (2m)) \cdot ln(4 / \delta))}$$

a concept drift alert is triggered, prompting the system to initiate incremental online model retraining on newly acquired network telemetry.

VIII. FUTURE DIRECTIONS AND CONCLUSION

Future research in machine learning intrusion detection will focus on three primary technical trajectories: (1) **Federated Collaborative Defense**: training shared threat models across decentralized organizations without exposing private packet telemetry [18]; (2) **Graph Machine Learning**: utilizing Graph Neural Networks (GNNs) to model multi-hop lateral movement and network topology context [19]; and (3) **Kernel-Bypass Hardware Offloading**: integrating pruned decision tree ensembles directly into eBPF/XDP drivers and programmable SmartNICs to support line-rate threat mitigation at sub-microsecond latencies [20].

In conclusion, Machine Learning represents an essential foundation for modern network intrusion detection. Tree-based ensemble models—particularly XGBoost and Random Forest—deliver an optimal balance of detection accuracy (exceeding 99%), low False Alarm Rates (0.4%), and microsecond inference latencies on commodity CPU hardware. By combining rigorous feature selection, adversarial hardening, and post-hoc TreeSHAP explainability, organizations can deploy practical, interpretable intrusion detection systems capable of defending enterprise networks against sophisticated cyber threats.

REFERENCES

[1] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153–1176, 2016.

[2] M. Roesch, "Snort: Lightweight intrusion detection for networks," in Proc. 13th USENIX Conf. on System Administration (LISA), 1999, pp. 229–238.

[3] C. F. Tsai, Y. F. Hsu, C. Y. Lin, and W. Y. Lin, "Intrusion detection by machine learning: A review," Expert Systems with Applications, vol. 36, no. 10, pp. 11994–12000, 2009.

[4] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[6] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," Computers & Security, vol. 86, pp. 147–167, 2019.

[7] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE CISDA, 2009, pp. 1–6.

[8] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems," in Proc. IEEE MilCIS, 2015, pp. 1–6.

[9] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new dataset for intrusion detection to enhance cybersecurity," in Proc. ICISSP, 2018, pp. 108–116.

[10] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an incomplete machine learning dataset: The case of CICIDS2017," in Proc. USENIX CSET, 2021, pp. 25–32.

[11] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[12] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf., 2016, pp. 785–794.

[14] H. Peng, F. Long, and C. Ding, "Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 27, no. 8, pp. 1226–1238, 2005.

[15] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, no. 1, pp. 56–67, 2020.

[16] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in Proc. ACM ASIACCS, 2017, pp. 506–519.

[17] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," ACM Computing Surveys, vol. 46, no. 4, pp. 1–37, 2014.

[18] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. AISTATS, 2017, pp. 1273–1282.

[19] Z. Wu et al., "A comprehensive survey on graph neural networks," IEEE Transactions on Neural Networks and Learning Systems, vol. 32, no. 1, pp. 4–24, 2020.

[20] D. Scholz et al., "Performance analysis of line-rate packet processing using eBPF and XDP," in Proc. IEEE NetSoft, 2020, pp. 124–131.

[21] R. Vinayakumar et al., "Evaluating deep learning approaches by comparing them with machine learning for network intrusion detection systems," IEEE Access, vol. 7, pp. 41525–41550, 2019.

[22] S. Guha and S. Roy, "Network intrusion detection using machine learning and deep learning: A comparative study," Journal of Network and Computer Applications, vol. 182, p. 103043, 2021.

[23] B. Ingre and A. Yadav, "Performance analysis of NSL-KDD dataset using ANN," in Proc. IEEE ICSPCom, 2015, pp. 92–96.

[24] M. A. Ferrag et al., "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," Journal of Information Security and Applications, vol. 50, p. 102419, 2020.

[25] J. Zhang, M. Zulkernine, and A. Haque, "Random-forests-based network intrusion detection systems," IEEE Transactions on Systems, Man, and Cybernetics, Part C, vol. 38, no. 5, pp. 649–659, 2008.